```
4.75
Normal division yields
the expected outcome
  >>> 19//4
4
Integer division dis-
cards the fraction (0.75)
flooring the result
  >>> 19//-4
-5
```

With negative numbers the rounding works as expected for negative numbers. i.e. -4.75 rounds to -5

Another popular mathematical operation which is natively supported by Python is the exponent operator. You can use it as follows:

```
  >>> 2**32
  4294967296
```

Python supports multiple simultaneous assignments:

```
  a = b = c = d = 0
  and
  a, b, c, d = 1, 2, 3, 4
```

Python can very easily operate on complex numbers. This is very useful for those wanting to use Python for mathematical applications. You need not write your own class or use a library, complex operations are inbuilt. Hence they also have a much more natural syntax. Complex numbers are written as 1 + 5j or 29 – 34J. So the following will work out-of-the-box:

```
  >>> (1+11J) + (9-13j)
  (10-2j)
  >>> (1+11J) - (9-13j)
  (-8+24j)
  >>> (1+11J) * (9-13j)
  (152+86j)
  >>> (1+11J) / (9-13j)
  (-0.536+0.448j)
```

Many other complex number operations such as extracting the real part, the complex part, getting the magnitude, etc are all supported. However let's move on.

Strings in Python

Strings in Python cannot be changed once assigned, unlike many other languages. So once you set a variable to a string value, you cannot manipulate the original string. As usual, you can express strings by enclosing them in double or single quotes. You can use quotes inside strings by using escaping the quotes (as in 'here\'s how to do it') or by using an alternate quote style (as in "here's how to do it"). If you have a multi-line string, you can use triple-quotes (''' or """) as:

```
  '''
Here
is some
multi-line text
'''
  Or you can use a raw string:
  somestring = r"Here
is some
multi-line text"
```

In a raw string, escaped entities such as '\n' for newline \"and \' are not converted but used as is.

String can be added:

```
  text = "Some" + "Text"
  or even multiplied:
  >>> text * 5
  'SomeTextSomeTextSome-
TextSomeTextSomeText'
```

Unlike some other languages, to find the length of a string in Python, you don't have something like string.length, instead you have to use the inbuilt length function as follows:

```
  len(string)
```

This function (len) will also work on other objects such as arrays and hashes that have lengths.

Python has some very powerful ways of working with strings. Python includes a syntax for accessing the elements of a list or string using the usual box [], however in Python this syntax is greatly empowered. In Python, the square brackets take three parameters [start:end:step]. So somestring[a:b:c] is equivalent to saying "Return a string composed of the characters of string 'somestring' where every cth starting from index a and going on till index b is chosen." Some or all of these parameters can be omitted. If you omit a starting index, then it will be taken as 0, if you skip the end index, it will be taken as the length of the string, if you omit the step, it will be taken as 1. All three of these can be negative as well! In case of negative indices, the counting will start at the end of the string, as will become evident from the examples that follow. Finally, you can have a negative step as well, in which case characters will be read in reverse, making reversing s string in Python as simple as somestring[::-1].

foo[2:11]	'is is a s'
foo[:11]	'This is a s'
foo[5:]	'is a sentence'
foo[1:-1]	'his is a sentenc'
foo[:-1]	'This is a sentenc'
foo[::2]	'Ti sasnec'
foo[::3]	'Tss nn'
foo[::-1]	'ecnetnes a si sihT'

```
  Let us say we have:
  foo = "This is a sentence"
  Then:
```

Lists in Python:

Lists are equivalent to arrays in many other languages. However in Python a list can have items of different types.

```
  a_list = ['reticulatus',
'molurus', 2, 3.1]
```

Lists support the same kind of operations with square brackets as we have explored with strings above, with the same [start:stop:step] syntax.

To add elements to an existing list, you can use list.append(newitem), or list.insert(newitem,index). You can also extend a list with another list using list.extend(anotherlist). Removing items is accomplished with list.pop(), which removes the last item, or list.pop(index), which removes the item at the index position.

To search in a list, you can use list.index(value), or if you don't need to know the index, you can simply do the following:

PYTHON HUMOUR

A little girl goes into a pet store and asks for a wabbit. The shop keeper looks down at her, smiles and says:
"Would you like a lovely fluffy little white rabbit, or a cutesy wootesly little brown rabbit?"
"Actually", says the little girl, "I don't think my python would notice."
—Nick Leaton, Wed, 04 Dec 1996